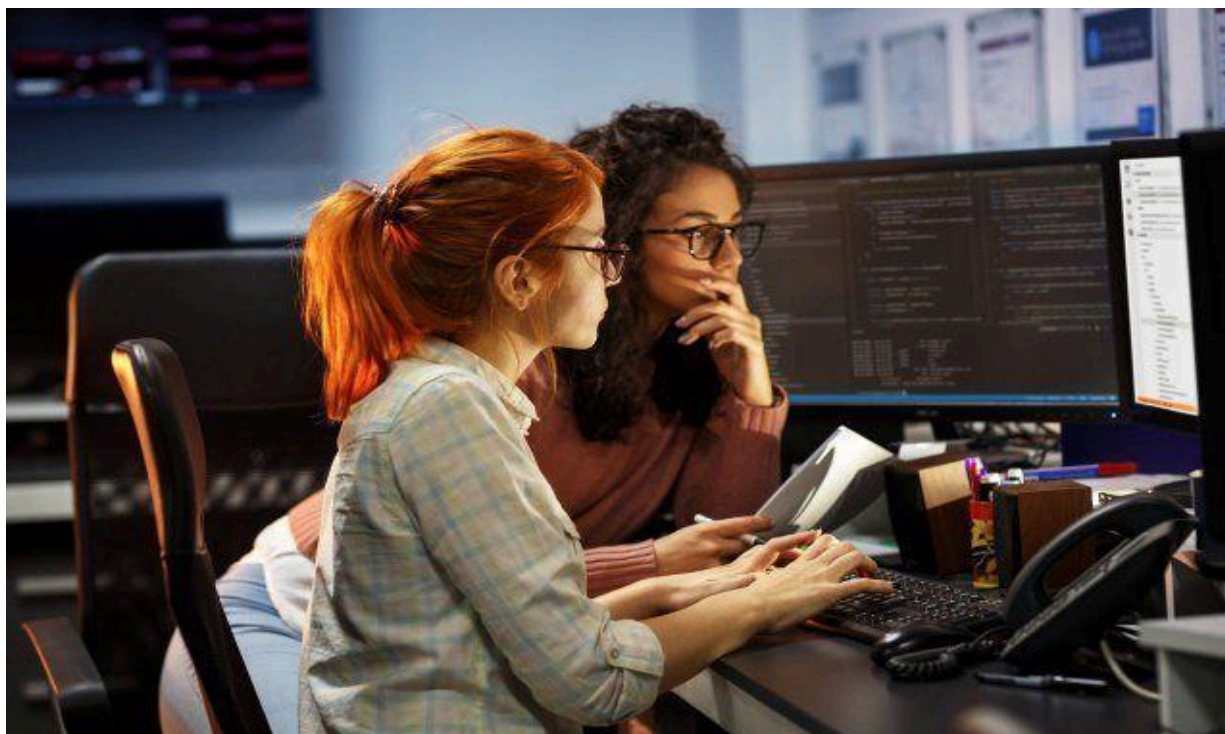




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura: Programación Orientada a Objetos	Unidad #: 1	Tema: Programación y Principios de Diseño Orientado a Objetos.
	SEMANA 4	Autor:

Introducción:

Bienvenidos al material de lectura #4 de la Unidad #1 de la materia Programación Orientada a Objetos. En este documento exploramos la introducción y los conceptos de constructores, métodos get/set y principios de diseño.

Este material ha sido diseñado para brindarte una base sólida en el ámbito del desarrollo de software, preparándote para afrontar los retos y oportunidades que surgirán en tu trayectoria profesional. A medida que avances en el contenido, te animamos a involucrarte activamente en las actividades propuestas, ya que contribuirán significativamente a tu proceso de aprendizaje.

Anota a continuación los aspectos que consideras más importantes de esta sección:

```

public Coche(String marca, String modelo, int anio) {
    System.out.println("Ejecutando el constructor...");
    this.marca = marca; // 'this' se refiere al objeto
actual
    this.modelo = modelo;
    this.anio = anio;
}

public void describir() {
    System.out.println("Coche: " + this.marca + " " +
this.modelo + " del año " + this.anio);
}
}

// --- Uso ---
// Al hacer "new Coche(...)", se llama al constructor.
Coche miDeportivo = new Coche("Ferrari", "F8 Tributo", 2024);
miDeportivo.describir(); // Salida: Coche: Ferrari F8 Tributo
del año 2024

```

Constructor por defecto: Si no defines ningún constructor en tu clase, Java proporciona uno automáticamente. Este es el "constructor por defecto", no tiene parámetros y deja los atributos con sus valores predeterminados (0 para números, `null` para objetos, `false` para booleanos). Sin embargo, en el momento en que defines *cualquier* constructor, Java ya no te regala el constructor por defecto.

1.9. Métodos `get` y `set`

Estos métodos son fundamentales para uno de los pilares de la POO: la **Encapsulación**. La encapsulación consiste en ocultar el estado interno de un

objeto (sus atributos) y exponer solo las operaciones necesarias para interactuar con él. Esto protege los datos de modificaciones indebidas y nos permite añadir lógica (como validaciones) al acceder a ellos.

Para lograrlo, los atributos se declaran como **private** y se crean métodos públicos para leerlos y modificarlos.

- **Métodos **get** (Getters):** Su única función es **obtener** o **leer** el valor de un atributo privado. Por convención, su nombre empieza con **get** seguido del nombre del atributo.
- **Métodos **set** (Setters):** Su función es **establecer** o **modificar** el valor de un atributo privado. Permiten añadir lógica de validación antes de realizar el cambio. Por convención, su nombre empieza con **set**.

Ejemplo continuando con la clase **Coche:**

```
public class Coche {  
    private String marca;  
    private int anio;  
  
    public Coche(String marca, int anio) {  
        this.marca = marca;  
        this.anio = anio;  
    }  
  
    // --- Getter para 'marca' ---  
    public String getMarca() {  
        return this.marca;  
    }  
  
    // --- Getter para 'anio' ---  
    public int getAnio() {
```

```
        return this.anio;
    }

    // --- Setter para 'anio' ---
    // Podemos agregar lógica de validación.
    public void setAnio(int nuevoAnio) {
        if (nuevoAnio > 1980 && nuevoAnio <= 2025) { //
Lógica de validación
            this.anio = nuevoAnio;
        } else {
            System.out.println("Error: El año " + nuevoAnio +
" no es válido.");
        }
    }
}

// --- Uso ---
Coche miCoche = new Coche("Ford", 2020);
System.out.println("Año actual: " + miCoche.getAnio()); //
Salida: 2020

miCoche.setAnio(2022); // El valor es válido, se cambia.
System.out.println("Año nuevo: " + miCoche.getAnio()); //
Salida: 2022

miCoche.setAnio(1970); // El valor no es válido.
// Salida: Error: El año 1970 no es válido.
```

```
System.out.println("Año después de intento fallido: " +  
miCoche.getAnio()); // Salida: 2022 (no cambió)
```

1.10. Principios de Diseño

Escribir código que funcione es solo el primer paso. Escribir código **limpio, mantenible, escalable y robusto** es lo que diferencia a un programador profesional. Los principios de diseño son guías y buenas prácticas que nos ayudan a lograrlo.

1.10.1. Cohesión, Acoplamiento, SRP y Modularidad

Estos cuatro conceptos están íntimamente relacionados y son la base de un buen diseño.

- **Cohesión (Alta Cohesión):** Mide qué tan enfocada está una clase en una sola tarea bien definida. **Una clase altamente cohesiva hace una sola cosa y la hace bien.** Por ejemplo, una clase **CalculadorDeImpuestos** solo debe calcular impuestos, no enviar emails ni guardar en base de datos.
 - **Acoplamiento (Bajo Acoplamiento):** Mide el grado de dependencia entre dos clases. **El bajo acoplamiento significa que un cambio en una clase no debería obligar a cambiar otras clases.** Si para cambiar la clase **Motor** necesitas modificar las clases **Rueda**, **Chasis** y **Volante**, tienes un alto acoplamiento (lo cual es malo).
 - **SRP (Single Responsibility Principle):** Es la materialización de la alta cohesión. Lo veremos en SOLID.
 - **Modularidad:** Es el resultado de diseñar un sistema con **alta cohesión** y **bajo acoplamiento**. El sistema se divide en módulos independientes y bien definidos que se pueden desarrollar, probar y reemplazar con un impacto mínimo en el resto del sistema. Piensa en un sistema de audio con componentes (módulos): puedes cambiar el amplificador sin tener que cambiar los altavoces.
-

1.10.2. Introducción y Aplicación de los Principios SOLID

SOLID es un acrónimo que representa cinco principios de diseño fundamentales para la POO. Seguir estos principios conduce a un software más comprensible, flexible y mantenible.

- **S - Single Responsibility Principle (Principio de Responsabilidad Única):**
 - "Una clase debe tener una, y solo una, razón para cambiar".
 - Esto es sinónimo de **alta cohesión**. Si una clase maneja la lógica de negocio de un **Usuario** y también su representación en HTML, tiene dos responsabilidades (dos razones para cambiar). Deberían ser dos clases separadas: **Usuario** y **VistaUsuario**.
- **O - Open/Closed Principle (Principio Abierto/Cerrado):**
 - "Las entidades de software (clases, módulos) deben estar abiertas para la extensión, pero cerradas para la modificación".
 - Significa que deberías poder añadir nueva funcionalidad sin cambiar el código existente. Esto se logra comúnmente usando interfaces y clases abstractas. En lugar de modificar una clase **CalculadorDeArea** con un **if** para cada nueva figura (**if circulo... else if cuadrado...**), la clase trabaja con una interfaz **Figura** y puedes añadir nuevas figuras (**Triangulo**, **Hexagono**) que implementen esa interfaz sin tocar **CalculadorDeArea**.
- **L - Liskov Substitution Principle (Principio de Sustitución de Liskov):**
 - "Los objetos de una clase derivada deben poder ser sustituidos por objetos de su clase base sin alterar el correcto funcionamiento del programa".
 - Si tienes una clase **Pajaro** con un método **volar()**, y creas una subclase **Pinguino** que hereda de **Pajaro**, tienes un problema. Un pingüino es un pájaro, pero no vuela. Si un método espera un **Pajaro** y le pasas un **Pinguino**, podría fallar al llamar a **volar()**. Esto viola el principio. La herencia debe tener sentido lógico y de comportamiento.
- **I - Interface Segregation Principle (Principio de Segregación de Interfaces):**
 - "Ningún cliente debe ser forzado a depender de métodos que no utiliza".
 - Es mejor tener muchas interfaces pequeñas y específicas que una sola interfaz grande y genérica. Si tienes una interfaz **Trabajador** con los métodos **trabajar()**, **comer()** y **dormir()**, y una clase **Robot** la implementa, estás forzando a **Robot** a tener un método **comer()** y **dormir()** que no tienen sentido para él. La solución es segregar la interfaz en **ITrabajador**, **IComedor**, etc.
- **D - Dependency Inversion Principle (Principio de Inversión de Dependencias):**
 - "Los módulos de alto nivel no deben depender de los módulos de bajo nivel. Ambos deben depender de abstracciones. Las

abstracciones no deben depender de los detalles. Los detalles deben depender de las abstracciones."

- En lugar de que tu clase `ReporteNegocio` (alto nivel) cree directamente un objeto `ImpresoraEpson` (bajo nivel), debería depender de una interfaz `Impresora`. Así, puedes pasarle una `ImpresoraEpson`, una `ImpresoraHP` o incluso una `ImpresoraPDF` sin cambiar la clase `ReporteNegocio`.

1.10.3. Buenas Prácticas de Diseño de Clases y Métodos

- **Mantén las cosas pequeñas:** Las clases y los métodos deben ser cortos y enfocados (SRP). Un método no debería tener más de 15-20 líneas.
- **Nombres descriptivos:** Usa nombres que revelen la intención. `calcularTotalFactura()` es mucho mejor que `calc()`.
- **DRY (Don't Repeat Yourself):** No repitas el mismo código en varios lugares. Si lo haces, encapsúlalo en un método y llama a ese método.
- **Favor de la composición sobre la herencia:** La herencia crea un acoplamiento fuerte. A menudo es más flexible "dar" a una clase una capacidad (composición, ej: un `Coche` tiene un `Motor`) que hacerla "ser" algo (herencia, ej: un `Coche` es un `Vehiculo`).

1.10.4. Patrones Básicos: Singleton, DTO, MVC

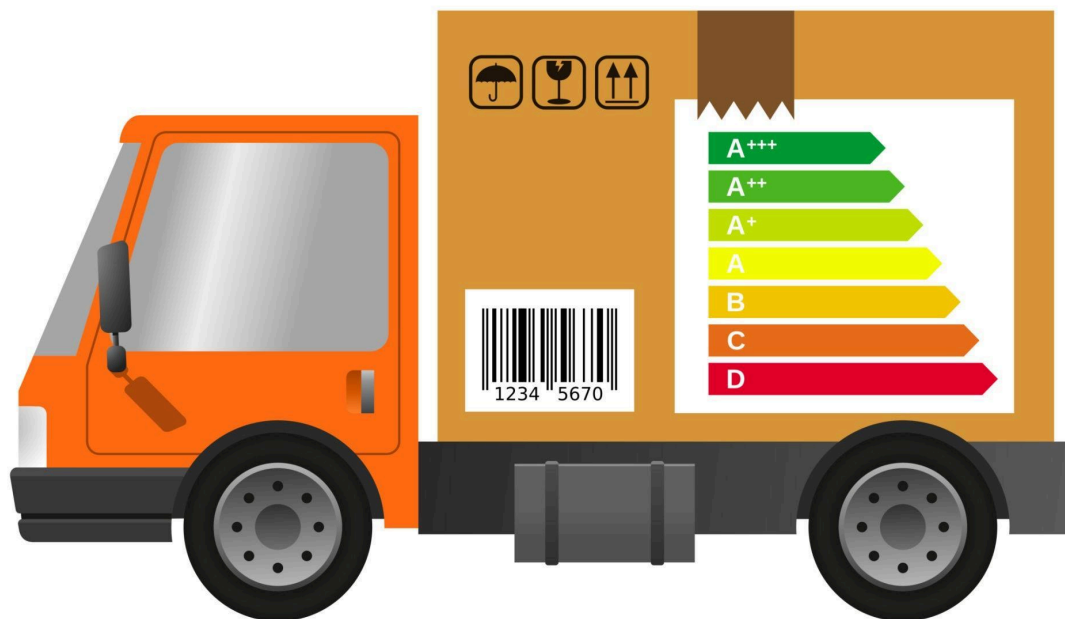
Los patrones de diseño son soluciones probadas y reutilizables a problemas comunes de diseño de software.

- **Singleton:**
 - **Propósito:** Garantizar que una clase tenga **una sola instancia** en toda la aplicación y proporcionar un punto de acceso global a ella.
 - **Uso:** Útil para gestionar recursos compartidos como una conexión a base de datos, un servicio de logging o la configuración de la aplicación.
 - **Implementación:** Se logra con un constructor privado, una variable estática privada para guardar la instancia y un método público (`getInstance()`) que crea la instancia si no existe y la devuelve.
- **DTO (Data Transfer Object):**

- **Propósito:** Es un objeto simple cuyo único fin es **transportar datos** entre diferentes capas o procesos de una aplicación (por ejemplo, entre el controlador y la vista).
- **Características:** No contiene lógica de negocio. Solo tiene atributos privados y sus métodos **get** y **set** públicos. Es como un "mensajero" tonto que solo carga y entrega datos.

Específicamente, pertenece a la categoría de los **patrones de arquitectura de software**, ya que su función principal es ayudar a estructurar la comunicación entre las diferentes capas de una aplicación.

La característica más importante de un DTO es que **no contiene lógica de negocio**. Es un "mensajero" tonto. No sabe cómo calcular, validar o guardar nada; su única responsabilidad es cargar los datos, moverse de un punto A a un punto B y entregar su carga.



Características Principales

- **Atributos Simples:** Contiene campos (variables de instancia) para almacenar los datos que se van a transferir.
- **Getters y Setters:** Proporciona métodos públicos (**get...** y **set...**) para acceder y modificar esos atributos.

- **Sin Lógica de Negocio:** No hay métodos que realicen cálculos, validaciones complejas o interacciones con la base de datos.
- **Serializable:** A menudo, los DTOs están diseñados para ser fácilmente "serializables", es decir, convertibles a formatos como JSON o XML para ser enviados a través de una red.

Ejemplo Práctico

Imagina que estás registrando un nuevo usuario a través de una API REST. La información viaja desde el cliente (navegador web) hasta tu servidor.

1. El cliente envía una petición con un JSON que contiene **nombre**, **email** y **contraseña**.
2. En tu servidor (backend), en lugar de manejar tres variables separadas, el framework automáticamente agrupa estos datos en un DTO.

Este sería el DTO en Java:

```
// UserRegistrationDTO.java

// Es una clase simple, a veces llamada POJO (Plain Old Java
Object).

public class UserRegistrationDTO {

    private String nombre;

    private String email;

    private String password;


    // Constructor vacío (a menudo necesario para los
frameworks)

    public UserRegistrationDTO() {}


    // Getters y Setters para cada atributo
```

```
public String getNombre() {  
    return nombre;  
}  
  
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}  
  
public String getEmail() {  
    return email;  
}  
  
public void setEmail(String email) {  
    this.email = email;  
}  
  
public String getPassword() {  
    return password;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}
```

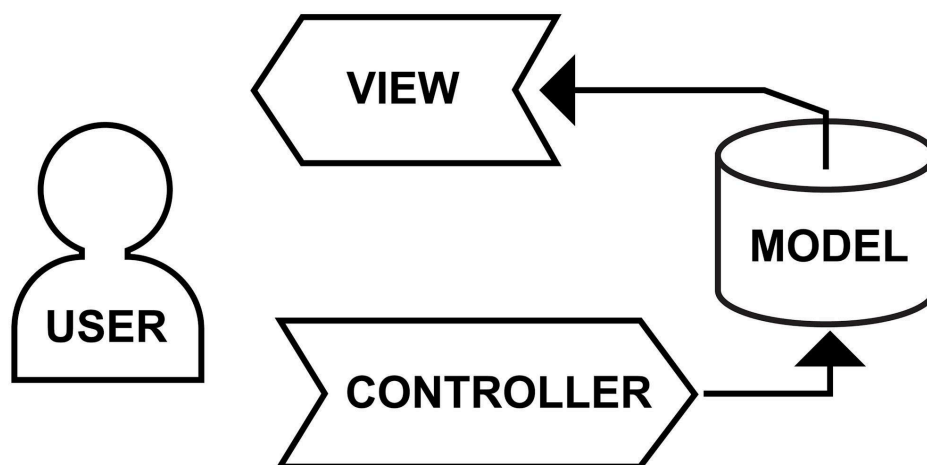
}

}

Luego, tu controlador recibiría un solo objeto `UserRegistrationDTO` y se lo pasaría a la capa de servicio para que esta sí aplique la lógica de negocio (validar los datos, crear el usuario real en la base de datos, etc.).

Al usar este patrón, mantienes tu código limpio, organizado y con un bajo acoplamiento entre las capas de la aplicación.

- **MVC (Model-View-Controller):**
 - **Propósito:** Es un patrón de arquitectura que **separa la lógica de una aplicación en tres partes interconectadas**, promoviendo el bajo acoplamiento y la responsabilidad única.
 - **Componentes:**
 - **Modelo (Model):** Representa los datos y la lógica de negocio. Es el "cerebro" de la aplicación. No sabe nada de la interfaz de usuario.
 - **Vista (View):** Es la representación de los datos del modelo, la interfaz de usuario (UI). Es la "cara" de la aplicación.
 - **Controlador (Controller):** Actúa como intermediario. Recibe la entrada del usuario (desde la Vista), la procesa, interactúa con el Modelo y actualiza la Vista.



MODEL - VIEW - CONTROLLER PATTERN